Personalized Networking Assistant:

AI-Powered Conversation Starter Generator

Project Description:

The Personalized Networking Assistant harnesses NLP and generative AI to reduce the anxiety and preparation burden of professional networking. The platform includes an Event Analyzer module, which extracts the key themes from an event description using zero-shot classification, a Fact Checker module, which retrieves a verified Wikipedia summary for each theme before any content is generated, and a Topic Generator, which produces natural-language conversation starters grounded in those facts, the attendee's bio, and their stated interests.

Using DistilBERT for theme extraction and GPT-2 Small for text generation, both running locally via Hugging Face Transformers, the assistant processes user input to deliver personalized, fact-checked conversation starters in under 20 seconds. Built with FastAPI and Streamlit and run entirely locally, the platform logs every session to history.json and every piece of feedback to feedback.json, giving users a simple, private, and fully functional demo without requiring any external accounts, API keys, or hosting.

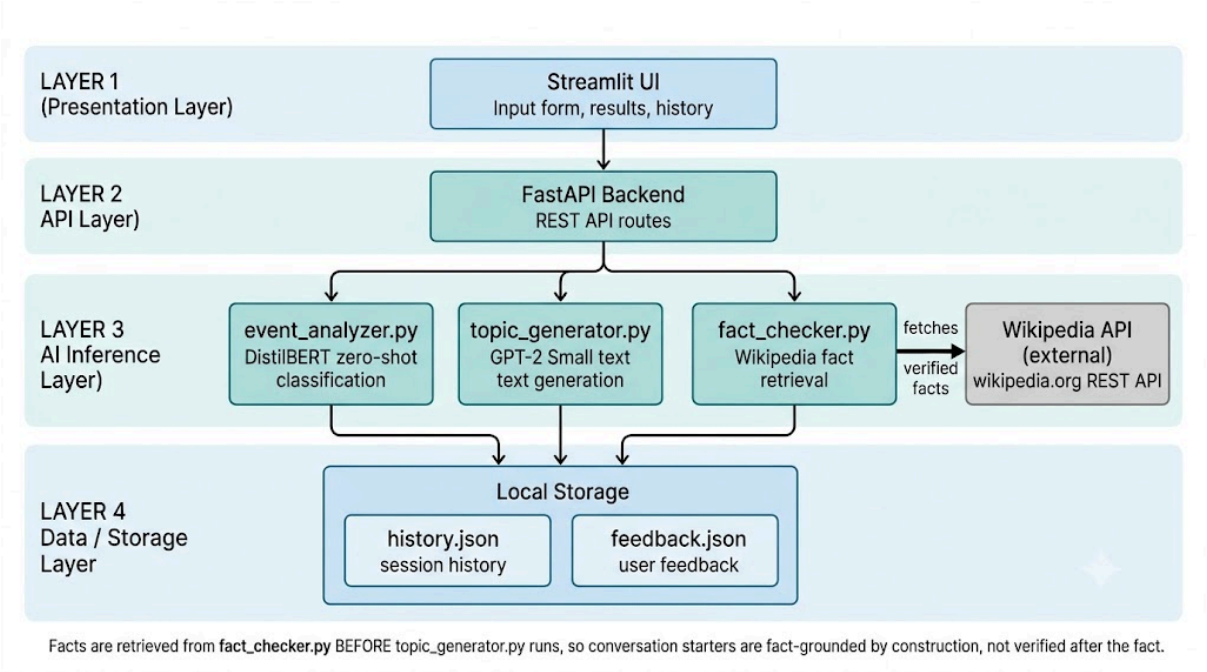
Scenarios

Scenario 1: A student attending a tech meetup on "AI applications in renewable energy and smart cities" enters the event description along with a short bio and interests ("machine learning, climate tech, urban planning"). The system extracts themes (technology, sustainability, leadership), retrieves a verified Wikipedia summary for each, and generates three conversation starters that reference real facts rather than generic small talk all in under 20 seconds.

Scenario 2: A user preparing for a blockchain conference wants to quickly understand a term they're unfamiliar with. They use the standalone Quick Fact-Check tool to search "blockchain in healthcare" independently of generating any starters, and receive a concise, verified summary pulled directly from Wikipedia.

Scenario 3: After attending an event and trying out a few starters, a user returns to the app, reviews their session history, and marks which starters actually worked well using thumbs up/down, building a personal record of what resonates for future events.

Technical Architecture



Description: The Personalized Networking Assistant follows a four-layer architecture that separates concerns cleanly: a Presentation Layer (Streamlit) collects bio, event description, and interests and displays results; an API Layer (FastAPI) exposes a small set of REST routes and orchestrates the pipeline; an AI Inference Layer contains three independent service modules (event_analyzer.py, fact_checker.py, topic_generator.py) plus the external Wikipedia integration; and a Data Layer persists everything to two local JSON files rather than a database, since the project is scoped for single-user local use.

The critical architectural decision in this project, caught and corrected mid-build rather than assumed correct from the start is that fact retrieval happens before generation, not after. Early drafts of the pipeline generated a starter first and treated fact-checking as a separate, optional step the user could run afterward. This meant a starter could be presented to the user before it was ever verified. The corrected design retrieves facts for each extracted theme first, then feeds those facts directly into the GPT-2 prompt as grounding context so a starter is fact-checked by construction, not as an afterthought.

Pre-requisites:

- **Python Programming Proficiency:** Python Documentation
- **FastAPI Framework Knowledge:** FastAPI Documentation
- **Streamlit Familiarity:** Streamlit Documentation
- **Hugging Face Transformers:** Transformers Documentation
- **Version Control with Git:** Git Documentation

- **Development Environment Setup:** Python 3.11+ (3.13 also verified working), pip, virtual environments(venv)

Project Workflow

Milestone 1: Model Selection and Architecture

- Activity 1.1: Research and select DistilBERT (zero-shot classification) and GPT-2 Small (text generation) as the two NLP models for the project.
- Activity 1.2: Define the four-layer application architecture, detailing the flow from Streamlit through FastAPI to the AI services and local storage.
- Activity 1.3: Set up the Python virtual environment and install dependencies (FastAPI, Streamlit, Transformers, requests, pytest, httpx).

Milestone 2: Core Functionalities Development

- Activity 2.1: Develop event_analyzer.py. Theme extraction from event descriptions using DistilBERT zero-shot classification against a predefined candidate label list.
- Activity 2.2: Develop fact_checker.py. Wikipedia REST API integration, including proper User-Agent headers (required by Wikipedia's API etiquette) and graceful fallback handling for topics with no matching page.
- Activity 2.3: Develop topic_generator.py. GPT-2 Small text generation, prompted with themes, bio, interests, and verified facts.
- Activity 2.4: Develop history_logger.py and feedback_logger.py. Local JSON persistence for session history and feedback.

Milestone 3: Main Application Logic

- Activity 3.1: Define Pydantic request/response schemas in `schemas.py` for type-safe validation across all API routes.
- Activity 3.2: Implement FastAPI routes in conversation.py: /analyze-event, /fact-check, /generate-conversation, and /feedback.
- Activity 3.3: Wire fact-grounding into the generation pipeline. This was a deliberate mid-build correction (see Technical Architecture above), changing /generate-conversation from calling event_analyzer → topic_generator directly to event_analyzer → fact_checker → topic_generator, so facts feed into generation rather than only being checked afterward.

Milestone 4: Frontend UI Using Streamlit

- Activity 4.1: Build the input form- bio, event description, and interests fields.
- Activity 4.2: Build the Generate + Display section, calling /generate-conversation and showing extracted themes and grounded starters.

- Activity 4.3: Build thumbs up/down feedback buttons, wired to the /feedback API endpoint (not a direct internal import, keeping the frontend cleanly separated from backend internals).
- Activity 4.4: Build the standalone Quick Fact-Check UI section, calling /fact-check independently of the generation flow.
- Activity 4.5: Build the Conversation History and Feedback History views, reading directly from the local JSON files.

Milestone 5: Testing and Local Deployment

- Activity 5.1: Write pytest tests covering event_analyzer, fact_checker, topic_generator, and the API routes (via FastAPI's TestClient, built on httpx).
- Activity 5.2: Deploy locally, no public deployment was needed or used, since the project is explicitly scoped for local single-user demonstration.
- Activity 5.3: Manual end-to-end verification through the Streamlit UI, confirming the full user journey works: generate → view themes/starters → give feedback → check history.

Milestone 6: Conclusion

Milestone 1: Model Selection and Architecture

In this milestone, the focus was selecting NLP models suited to two distinct tasks- classifying an event description into themes, and generating natural language starters while keeping both runnable locally on CPU, since the project has no GPU requirement or budget for hosted inference APIs.

Activity 1.1: Model Selection

- DistilBERT (typeform/distilbert-base-uncased-mnli)was selected for zero-shot classification: it lets the event description be scored against a predefined list of candidate themes (technology, sustainability, healthcare, etc.) without any task-specific fine-tuning.
- GPT-2 Small was selected for text generation via the Hugging Face text-generation pipeline, small enough to run on CPU within acceptable latency for a local demo, at the cost of sometimes-generic output quality (a documented, tested limitation, not a hidden one).

Activity 1.2: Application Architecture

Presentation Layer (Streamlit)

↓↑

API Layer (FastAPI) — routes: /analyze-event, /fact-check, /generate-conversation, /feedback

↓↑

AI Inference Layer — event_analyzer.py | fact_checker.py | topic_generator.py

↓↑

Data Layer — history.json, feedback.json

Activity 1.3: Environment Setup

python -m venv venv

venv\Scripts\Activate.ps1

pip install -r requirements.txt

requirements.txt includes: fastapi, uvicorn, streamlit, transformers, torch, requests, pydantic, pytest, httpx, wikipedia-api.

Milestone 2: Core Functionalities Development

Activity 2.1: event_analyzer.py

```
from transformers import pipeline

from app.config import MODEL_NAMES

classifier = pipeline("zero-shot-classification",
model=MODEL_NAMES["event_analysis"])

CANDIDATE_LABELS = [

    "artificial intelligence", "healthcare", "blockchain", "education",

    "sustainability", "finance", "technology", "leadership",

    "entrepreneurship", "climate change"

]

def extract_event_themes(description: str, candidate_labels=None):

    if candidate_labels is None:
```

```
        candidate_labels = CANDIDATE_LABELS

    result = classifier(description, candidate_labels)

    return result["labels"][:3]
```

Activity 2.2: fact_checker.py

A real early bug caught here: the first version made unauthenticated requests to Wikipedia's REST API, which silently failed every call ("Fact-checking failed" for every query). Wikipedia's API requiresa User-Agent header identifying the requester.

The fix:

```
import requests

from urllib.parse import quote

from app.config import FACT_CHECK_API

HEADERS = {"User-Agent": "PersonalizedNetworkingAssistant/1.0 (student
capstone project)"}

def fact_check(query: str) -> str:

    try:

        url = f"{FACT_CHECK_API}/{quote(query)}"

        response = requests.get(url, headers=HEADERS, timeout=10)

        if response.status_code != 200:

            return "No summary found."

        data = response.json()

        return data.get("extract", "No summary found.")

    except Exception:

        return "Fact-checking failed."

def fact_check_many(queries: list[str]) -> list[str]:
```

```
    return [fact_check(q) for q in queries]
```

Activity 2.3: topic_generator.py

A second real bug caught here: an early version asked GPT-2 to generate a numbered list of 3 starters in a single call, which produced either infinite repetition loops or an inconsistently-numbered blob that couldn't be parsed apart. The fix was to generate 3 independent completions instead of asking the model to self-organize a list:

```
import re

from transformers import pipeline, set_seed

from app.config import MODEL_NAMES

generator = pipeline("text-generation",
model=MODEL_NAMES["text_generator"])

set_seed(42)

def generate_topics(event_themes, user_interests, facts=None,
bio=None):

    facts = facts or []

    valid_facts = [f for f in facts if f and "failed" not in f.lower()
and "no summary" not in f.lower()]

    fact_snippet = valid_facts[0][:200] if valid_facts else ""

    bio_snippet = f"About me: {bio}. " if bio else ""

    prompt = (

        f"I'm at a networking event about {', '.join(event_themes)}. "

        f"I'm interested in {', '.join(user_interests)}. "

        f"{bio_snippet}"

        f"Fact: {fact_snippet} "

        f"A good conversation starter I could say is:"
```

```
    )
    outputs = generator(
        prompt,
        max_new_tokens=40,
        num_return_sequences=3,
        do_sample=True,
        temperature=0.9,
        top_p=0.92,
        repetition_penalty=1.3,
        no_repeat_ngram_size=3,
        return_full_text=False,
        pad_token_id=generator.tokenizer.eos_token_id,
    )
    suggestions = []
    for out in outputs:
        text = out["generated_text"].strip()
        match = re.search(r"^(.*?[.!?])", text)  # cut at first full
sentence
        cleaned = match.group(1) if match else text
        if cleaned:
            suggestions.append(cleaned)

    return suggestions if suggestions else ["Could not generate a
starter this time — try again."]
```

Activity 2.4: history_logger.py and feedback_logger.py

Simple JSON read/append/write helpers, each scoped to one file and one responsibility.

history_logger.py:

```
import json

from datetime import datetime

from pathlib import Path

HISTORY_FILE = Path("history.json")

def log_conversation(data: dict):

    data["timestamp"] = datetime.now().isoformat()

    if HISTORY_FILE.exists():

        with open(HISTORY_FILE, "r") as f:

            history = json.load(f)

    else:

        history = []

    history.append(data)

    with open(HISTORY_FILE, "w") as f:

        json.dump(history, f, indent=2)

def load_history():

    if HISTORY_FILE.exists():

        with open(HISTORY_FILE, "r") as f:

            return json.load(f)

    return []
```

feedback_logger.py :

```python
import json

from datetime import datetime

from pathlib import Path

FEEDBACK_FILE = Path("feedback.json")

def log_feedback(suggestion: str, action: str):

    entry = {

        "suggestion": suggestion,

        "feedback": action,

        "timestamp": datetime.now().isoformat()

    }

    if FEEDBACK_FILE.exists():

        with open(FEEDBACK_FILE, "r") as f:

            data = json.load(f)

    else:

        data = []

    data.append(entry)

    with open(FEEDBACK_FILE, "w") as f:

        json.dump(data, f, indent=2)

def get_feedback():

    if FEEDBACK_FILE.exists():

        with open(FEEDBACK_FILE, "r") as f:

            return json.load(f)

    return []
```

Milestone 3: Main Application Logic

Activity 3.1 & 3.2: Schemas and Routes

```python
# app/routers/conversation.py

from fastapi import APIRouter

from app.models.schemas import (

    EventInput, ConversationRequest, FactCheckRequest,

    ConversationResponse, FactCheckResponse

)

from app.services import event_analyzer, topic_generator, fact_checker,
history_logger, feedback_logger



router = APIRouter()



@router.post("/analyze-event")

def analyze_event(data: EventInput):

    themes = event_analyzer.extract_event_themes(data.description)

    return {"topics": themes}



@router.post("/fact-check", response_model=FactCheckResponse)

def fact_check(data: FactCheckRequest):

    # Standalone quick fact-check — independent of starter generation

    summary = fact_checker.fact_check(data.query)

    return FactCheckResponse(summary=summary)
```

```python
@router.post("/generate-conversation",
response_model=ConversationResponse)
def generate_conversation(data: ConversationRequest):
    # 1. Extract themes from the event description
    themes = event_analyzer.extract_event_themes(data.description)

    # 2. Fetch verified facts for those themes BEFORE generating —
    #    this is the grounding step, so the starter is fact-checked
    #    by construction rather than verified after the fact
    facts = fact_checker.fact_check_many(themes)

    # 3. Generate starters using themes + interests + facts as context
    suggestions = topic_generator.generate_topics(themes,
data.interests, facts, data.bio)
    # 4. Log the full session
    history_logger.log_conversation({
        "description": data.description,
        "interests": data.interests,
        "topics": themes,
        "facts": facts,
        "suggestions": suggestions,
        "bio": data.bio
    })
```

```
    return ConversationResponse(topics=themes, facts=facts,
suggestions=suggestions)



@router.post("/feedback")

def submit_feedback(suggestion: str, action: str):

    feedback_logger.log_feedback(suggestion, action)

    return {"status": "recorded"}
```

Activity 3.3: The Grounding Fix

The single most important design decision in this project. An early version of this route called topic_generator.generate_topics() directly after theme extraction, with fact-checking exposed only as a separate route the user could optionally call afterward. This meant the starter shown to the user was never actually guaranteed to be fact-grounded — verification, if it happened at all, happened after the user already had the (possibly ungrounded) text. The corrected order above — themes → facts → generation, with facts passed directly into the prompt makes every generated starter fact-checked by construction.

Milestone 4: Frontend UI Using Streamlit

Activity 4.1 Input Section:

Bio, event description, and interests collected via st.text_area and st.text_input.

Activity 4.2 Generate + Display:

On button click, POSTs to /generate-conversation, stores the response in st.session_state (so results persist across the reruns that Streamlit triggers on every widget interaction), and displays extracted themes plus each generated starter.

Activity 4.3 Feedback Buttons:

Thumbs up/down per starter, each calling POST /feedback deliberately over HTTP rather than importing feedback_logger directly into the frontend, to keep the presentation layer

cleanly separated from backend internals (an issue that was actually introduced once, caught, and fixed during the build, including a duplicate-write bug where both a direct import call and the API call were firing on the same click).

Activity 4.4 Quick Fact-Check Section:

An independent text input and button calling /fact-check, decoupled entirely from the generation flow, supports the "quick research" use case rather than acting as a post-hoc check on generated content.

Activity 4.5 History and Feedback History Views:

Both read directly from history.json and feedback.json on disk, a deliberate, reasonable exception to the "frontend only talks to the API" rule, since these are read-only displays rather than state-changing actions.

Milestone 5: Testing and Local Deployment

Activity 5.1: Automated Testing

pytest tests/ -v

5 tests covering event_analyzer, fact_checker, topic_generator, and both /generate-conversation and /fact-check routes (via FastAPI's TestClient, itself built on httpx), all passing, calling the real models and a live Wikipedia endpoint rather than mocks, so a pass genuinely proves the pipeline works end-to-end, not just that the logic is internally consistent.

Activity 5.2: Local Deployment

 # Terminal 1: uvicorn app.main:app --reload

 # Terminal 2: streamlit run frontend/streamlit_app.py

- FastAPI: http://127.0.0.1:8000/docs
- Streamlit: http://localhost:8501

No public deployment was used or needed, the project is explicitly scoped for local, single-user demonstration per the Solution Requirements.

Activity 5.3: Manual Verification

A full walkthrough of the actual user journey (enter details → generate → view themes/facts/starters → give feedback → check history → check feedback history → run a standalone fact-check) was performed and confirmed working end-to-end before considering the build complete.

Exploring the Application

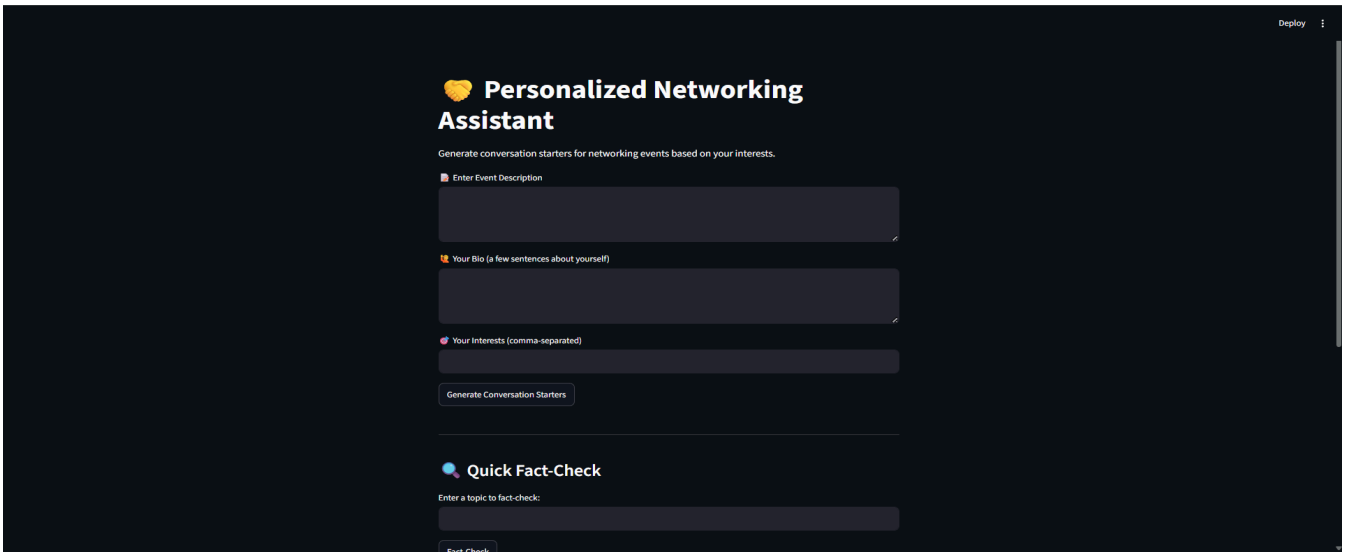
Input Section: A simple three-field form (bio, event description, interests) with a single "Generate Conversation Starters" button.

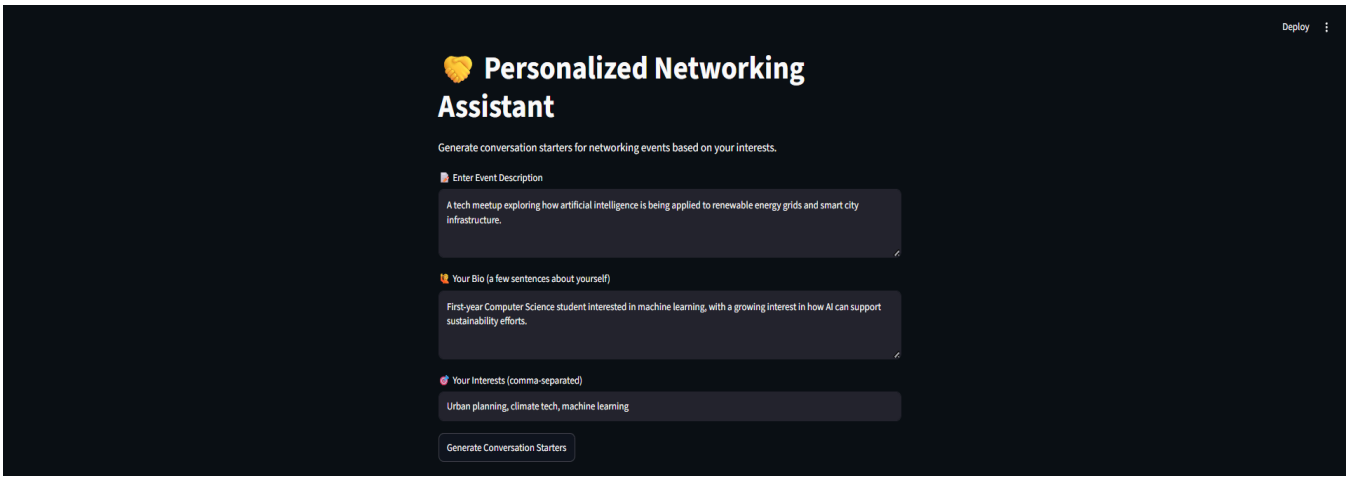
Results Section: Displays extracted themes as a list, followed by each generated conversation starter with 👍/👎 buttons beneath it.

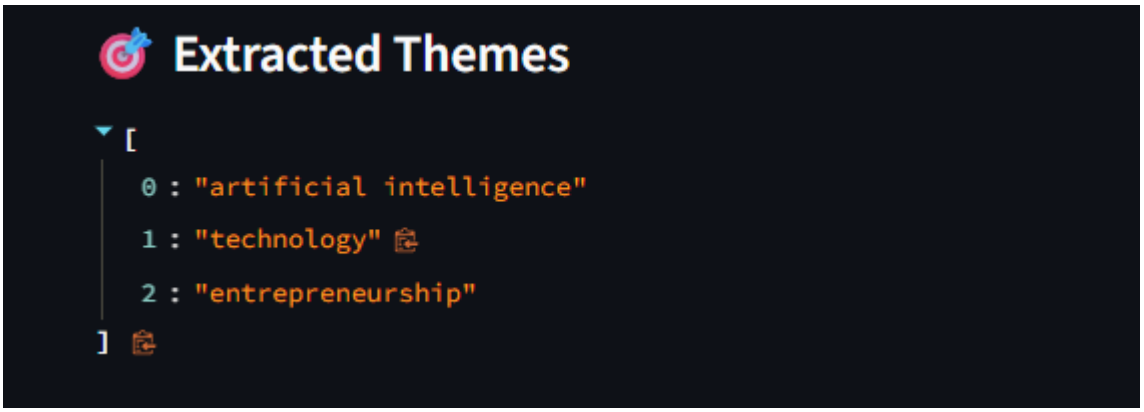
Quick Fact-Check Section: A standalone text input and button, independent of the generation flow, for looking up any topic directly.

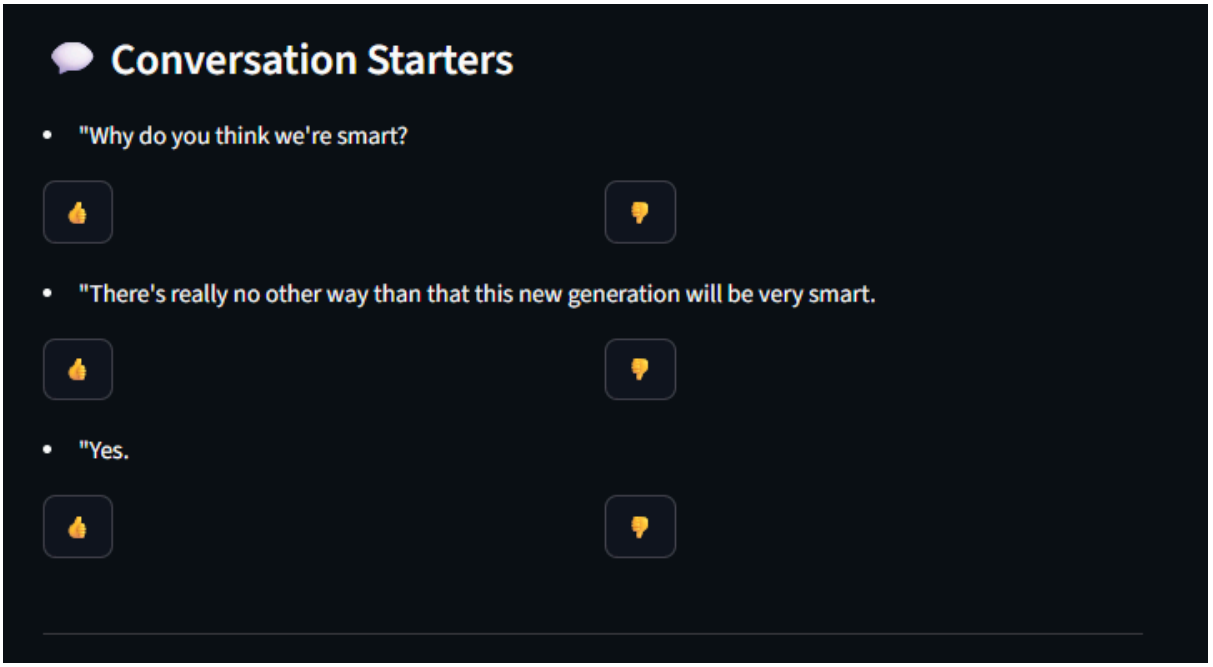
History Section: Shows the 5 most recent sessions — event description, interests, extracted topics, and generated suggestions, each timestamped.

Feedback History Section: Shows the 10 most recent feedback entries with a 👍/👎 icon, the starter text, and a timestamp.


Deploy
🤝 Personalized Networking Assistant
Generate conversation starters for networking events based on your interests.
Enter Event Description
Your Bio (a few sentences about yourself)
Your Interests (comma-separated)
Generate Conversation Starters
🔍 Quick Fact-Check
Enter a topic to fact-check:


Deploy
🤝 Personalized Networking Assistant
Generate conversation starters for networking events based on your interests.
Enter Event Description
A tech meetup exploring how artificial intelligence is being applied to renewable energy grids and smart city infrastructure.
Your Bio (a few sentences about yourself)
First-year Computer Science student interested in machine learning, with a growing interest in how AI can support sustainability efforts.
Your Interests (comma-separated)
Urban planning, climate tech, machine learning
Generate Conversation Starters


Extracted Themes
[
0 : "artificial intelligence"
1 : "technology"
2 : "entrepreneurship"
]


Conversation Starters
"Why do you think we're smart?
"There's really no other way than that this new generation will be very smart.
"Yes.


Quick Fact-Check
Enter a topic to fact-check:
Fact-Check

🔍 Quick Fact-Check

Enter a topic to fact-check:

Quantum Computing

Fact-Check

A quantum computer is a real or theoretical computer that exploits quantum phenomena like superposition and entanglement in an essential way. It is widely believed that a quantum computer could perform some calculations exponentially faster than any classical computer. For example, a large-scale quantum computer could break some widely used encryption schemes and aid physicists in performing physical simulations. However, current hardware implementations of quantum computation are largely experimental and only suitable for specialized tasks.

📜 View Previous Conversations

Show History

🕐 2026-07-04T19:50:25.530228

Event: A tech meetup exploring how artificial intelligence is being applied to renewable energy grids and smart city infrastructure.

Interests: Urban planning, climate tech, machine learning

Topics: artificial intelligence, technology, entrepreneurship

Suggestions:

- "Why do you think we're smart?
- "There's really no other way than that this new generation will be very smart.
- "Yes.

🕐 2026-07-04T17:49:41.703789

Event: A tech meetup exploring how artificial intelligence is being applied to renewable energy grids and smart city infrastructure.

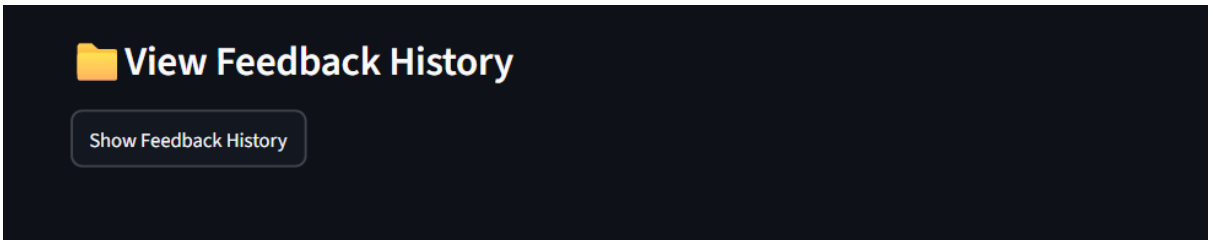
Interests: Urban planning, climate tech, machine learning

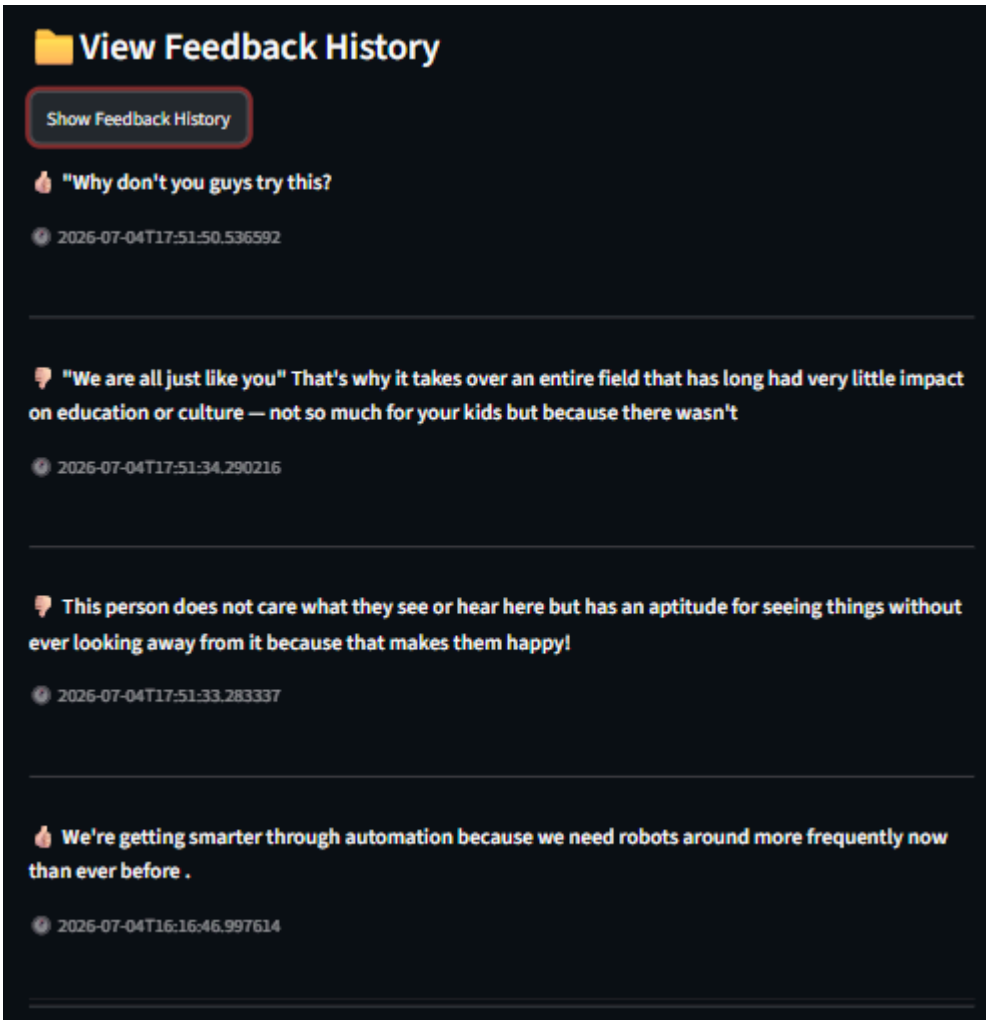
Topics: artificial intelligence, technology, entrepreneurship

Suggestions:

- "Why don't you guys try this?
- This person does not care what they see or hear here but has an aptitude for seeing things without ever looking away from it because that makes them happy!
- "We are all just like you" That's why it takes over an entire field that has long had very little impact on education or culture — not so much for your kids but because there wasn't

---


View Feedback History
Show Feedback History


View Feedback History
Show Feedback History
"Why don't you guys try this?
2026-07-04T17:51:50.536592
"We are all just like you" That's why it takes over an entire field that has long had very little impact on education or culture — not so much for your kids but because there wasn't
2026-07-04T17:51:34.290216
This person does not care what they see or hear here but has an aptitude for seeing things without ever looking away from it because that makes them happy!
2026-07-04T17:51:33.283337
We're getting smarter through automation because we need robots around more frequently now than ever before .
2026-07-04T16:16:46.997614

Conclusion

The Personalized Networking Assistant is a locally-run AI system that combines zero-shot classification, fact grounding, and text generation into a single coherent pipeline — one where the most important design decision (retrieving facts *before* generation rather than after) was caught and corrected mid-build rather than assumed correct from the start. Built with FastAPI and Streamlit, the project demonstrates a complete, tested, end-to-end user journey: from raw event description to a fact-checked conversation starter, with session history and feedback logging throughout.

Looking ahead, the project's own Brainstorming & Idea Prioritization (Folder 1) already identifies clear directions for future work explicitly deferred from this build: a live AI chat companion usable during the event itself, an attendee-to-attendee matching algorithm, multi-language support, and tone/formality adjustment for different event types. By continuing to refine the underlying prompts and potentially fine-tuning GPT-2 or swapping in a larger instruction-tuned model, the platform is well-positioned to move from a solid single-user local demo toward a genuinely deployable networking tool.